

```
!pip install -q transformers datasets evaluate sentencepiece sacrebleu rouge-score scikit-learn
```

```
Preparing metadata (setup.py) ... done
84.1/84.1 kB 3.5 MB/s eta 0:00:00
100.8/100.8 kB 7.4 MB/s eta 0:00:00
Building wheel for rouge-score (setup.py) ... done
```

```
import torch
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from datasets import Dataset
from evaluate import load
from sklearn.model_selection import train_test_split
from google.colab import files
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cpu
```

```
data = {
    "question": [
        "What is KYC?",
        "How to open a bank account?",
        "Can I check my balance online?",
        "What is an ATM?",
        "How to reset ATM PIN?",
        "What is net banking?",
        "How to transfer money online?",
        "What is a debit card?",
        "What is a credit card?",
        "How to apply for a loan?",
        "What is interest rate?",
        "Can I open account online?",
        "What documents are required for KYC?",
        "How to block lost ATM card?",
        "What is minimum balance?",
        "What is IFSC code?",
        "How to update mobile number?",
        "What is UPI?",
        "Is online banking safe?",
        "How to activate debit card?"
    ],
    "answer": [
        "KYC means Know Your Customer and is used to verify identity.",
        "You can open a bank account by visiting the branch with valid ID proof.",
        "Yes you can check your balance using net banking or mobile app.",
        "ATM is a machine used to withdraw money and check balance.",
        "You can reset your ATM PIN using ATM machine or mobile banking.",
        "Net banking is an online service to access your bank account.",
        "You can transfer money using UPI or net banking.",
        "A debit card allows you to withdraw money from your account.",
        "A credit card allows you to borrow money from the bank.",
        "You can apply for a loan online or by visiting the bank.",
        "Interest rate is the percentage charged on borrowed money.",
        "Yes many banks allow online account opening.",
        "You need ID proof and address proof.",
        "Contact bank customer care immediately to block your card.",
        "Minimum balance is the amount you must maintain in your account.",
        "IFSC code is used to identify bank branches for transactions.",
        "Visit bank branch or use net banking to update number.",
        "UPI is a system for instant money transfer using mobile.",
        "Yes it is safe if you follow security guidelines.",
        "You can activate it at ATM or through mobile banking."
    ]
}

df = pd.DataFrame(data)
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

Using device: cpu

```
train_q, test_q, train_a, test_a = train_test_split(
    df["question"], df["answer"], test_size=0.3, random_state=42
)

train_dataset = Dataset.from_dict({"question": list(train_q), "answer": list(train_a)})
test_dataset = Dataset.from_dict({"question": list(test_q), "answer": list(test_a)})
```

```
model_name = "google/mt5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% 553/553 [00:00<00:00, 30.6kB/s]

tokenizer_config.json: 100% 82.0/82.0 [00:00<00:00, 10.0kB/s]

spiece.model: 100% 4.31M/4.31M [00:00<00:00, 40.7MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
special_tokens_map.json: 100% 99.0/99.0 [00:00<00:00, 11.0kB/s]

pytorch_model.bin: 100% 1.20G/1.20G [00:10<00:00, 136MB/s]

model.safetensors: 58% 691M/1.20G [00:06<00:04, 118MB/s]

Loading weights: 100% 192/192 [00:01<00:00, 179.06it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but both are p
The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but both are p
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the
generation_config.json: 100% 147/147 [00:00<00:00, 3.63kB/s]
```

```
max_len = 128

def preprocess(batch):
    inputs = tokenizer(batch["question"], padding="max_length", truncation=True, max_length=max_len)
    targets = tokenizer(batch["answer"], padding="max_length", truncation=True, max_length=max_len)

    labels = [[(t if t != tokenizer.pad_token_id else -100) for t in seq] for seq in targets["input_ids"]]
    inputs["labels"] = labels
    return inputs

train_tokenized = train_dataset.map(preprocess, batched=True)
test_tokenized = test_dataset.map(preprocess, batched=True)

train_tokenized.set_format("torch")
test_tokenized.set_format("torch")
```

```
Map: 100% 14/14 [00:00<00:00, 172.60 examples/s]

Map: 100% 6/6 [00:00<00:00, 233.56 examples/s]
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)

model.train()
for epoch in range(5):
    total_loss = 0

    for batch_item in train_tokenized:
        inputs = {
            "input_ids": batch_item["input_ids"].unsqueeze(0).to(device),
            "attention_mask": batch_item["attention_mask"].unsqueeze(0).to(device),
            "labels": batch_item["labels"].unsqueeze(0).to(device),
        }
```

```

optimizer.zero_grad()
outputs = model(**inputs)
loss = outputs.loss

loss.backward()
optimizer.step()

total_loss += loss.item()

print(f"Epoch {epoch+1} Loss: {total_loss/len(train_tokenized):.4f}")

```

```

Epoch 1 Loss: 24.3754
Epoch 2 Loss: 25.7792
Epoch 3 Loss: 28.4159
Epoch 4 Loss: 25.4379
Epoch 5 Loss: 20.7154

```

```

model.eval()

rouge = load("rouge")
bleu = load("bleu")

predictions = []
references = []

with torch.no_grad():
    for batch in test_tokenized:
        input_ids = batch["input_ids"].unsqueeze(0).to(device)

        outputs = model.generate(input_ids, max_new_tokens=50)

        pred = tokenizer.decode(outputs[0], skip_special_tokens=True)
        ref = tokenizer.decode([x for x in batch["labels"].tolist() if x != -100], skip_special_tokens=True)

        predictions.append(pred)
        references.append(ref)

```

```

Downloading builder script:      6.14k/? [00:00<00:00, 549kB/s]
Downloading builder script:      5.94k/? [00:00<00:00, 439kB/s]
Downloading extra modules:                               4.07k/? [00:00<00:00, 481kB/s]
Downloading extra modules:      3.34k/? [00:00<00:00, 311kB/s]

```

```

print("ROUGE:", rouge.compute(predictions=predictions, references=references))
print("BLEU:", bleu.compute(predictions=predictions,
                             references=[[r] for r in references]))

```

```

ROUGE: {'rouge1': np.float64(0.0196078431372549), 'rouge2': np.float64(0.0), 'rougeL': np.float64(0.0196078431372549), 'rougeLsu
BLEU: {'bleu': 0.0, 'precisions': [0.0, 0.0, 0.0, 0.0], 'brevity_penalty': 0.47802356999411627, 'length_ratio': 0.57534246575342

```

```

for i in range(3):
    print("Q:", test_q.iloc[i])
    print("Pred:", predictions[i])
    print("Actual:", test_a.iloc[i])
    print("-"*50)

```

```

Q: What is KYC?
Pred: <extra_id_0>
Actual: KYC means Know Your Customer and is used to verify identity.
-----

```

```

Q: What is UPI?
Pred: <extra_id_0>
Actual: UPI is a system for instant money transfer using mobile.
-----

```

```

Q: What is IFSC code?
Pred: <extra_id_0>
Actual: IFSC code is used to identify bank branches for transactions.
-----

```

